Lista dwukierunkowa

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 DoublyLinkedList Class Reference . . . 5

3.1.1 Detailed Description . . . 5

3.2 Node Class Reference . . . 5

3.2.1 Detailed Description . . . 6

3.2.2 Constructor & Destructor Documentation . . . 6

3.2.2.1 Node() . . . 6

3.3 NodeFactory Class Reference . . . 6

3.3.1 Detailed Description . . . 7

3.3.2 Member Function Documentation . . . 7

3.3.2.1 createNode() . . . 7

4 File Documentation 9

4.1 DoublyLinkedList.cpp File Reference . . . 9

4.1.1 Detailed Description . . . 9

4.2 DoublyLinkedList.h File Reference . . . 9

4.2.1 Detailed Description . . . 9

4.3 DoublyLinkedList.h . . . 10

4.4 main.cpp File Reference . . . 10

4.4.1 Detailed Description . . . 10

4.5 Node.cpp File Reference . . . 10

4.5.1 Detailed Description . . . 11

4.6 Node.h File Reference . . . 11

4.6.1 Detailed Description . . . 11

4.7 Node.h . . . 11

4.8 NodeFactory.cpp File Reference . . . 11

4.8.1 Detailed Description . . . 11

4.9 NodeFactory.h File Reference . . . 12

4.9.1 Detailed Description . . . 12

4.10 NodeFactory.h . . . 12

Index 13

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DoublyLinkedList
: Klasa obsługująca pełny zestaw operacji na liście dwukierunkowej 5

Node
: Klasa przechowująca wartość oraz wskaźniki na sąsiednie węzły 5

NodeFactory
: Klasa odpowiedzialna za kontrolowane tworzenie nowych obiektów węzłów 6

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

DoublyLinkedList.cpp
: Implementacja pełnych operacji dla klasy DoublyLinkedList 9

DoublyLinkedList.h
: Definicja klasy zarządzającej listą dwukierunkową wraz ze wszystkimi operacjami 9

main.cpp
: Kompleksowe testy wszystkich wymaganych operacji na liście 10

Node.cpp
: Implementacja konstruktora klasy Node 10

Node.h
: Definicja klasy Node reprezentującej pojedynczy element listy dwukierunkowej 11

NodeFactory.cpp
: Implementacja metod klasy NodeFactory 11

NodeFactory.h
: Definicja klasy NodeFactory implementującej wzorzec projektowy Factory 12

Chapter 3

Class Documentation

3.1 DoublyLinkedList Class Reference

Klasa obsługująca pełny zestaw operacji na liście dwukierunkowej.

`#include <DoublyLinkedList.h>`

Public Member Functions

- void **prepend** (int val)
- void **append** (int val)
- void **insertAtIndex** (int index, int val)
- void **removeHead** ()
- void **removeTail** ()
- void **removeAtIndex** (int index)
- void **displayForward** ()
- void **displayBackward** ()
- void **displayNext** (Node *node)
- void **displayPrev** (Node *node)
- void **clear** ()
- Node * **getNode** (int index)

3.1.1 Detailed Description

Klasa obsługująca pełny zestaw operacji na liście dwukierunkowej.

The documentation for this class was generated from the following files:

- DoublyLinkedList.h
- DoublyLinkedList.cpp

3.2 Node Class Reference

Klasa przechowująca wartość oraz wskaźniki na sąsiednie węzły.

`#include <Node.h>`

Public Member Functions

- Node (int val)

  Konstruktor inicjalizujący węzeł podaną wartością.

Public Attributes

- int **data**

  Wartość przechowywana w węźle.
- Node * **next**

  Wskaźnik na następny węzeł
- Node * **prev**

  Wskaźnik na poprzedni węzeł

3.2.1 Detailed Description

Klasa przechowująca wartość oraz wskaźniki na sąsiednie węzły.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Node()

```
Node::Node (
            int val)
```

Konstruktor inicjalizujący węzeł podaną wartością.

Parameters

val	Liczba całkowita zapisywana w węźle.

The documentation for this class was generated from the following files:

- Node.h
- Node.cpp

3.3 NodeFactory Class Reference

Klasa odpowiedzialna za kontrolowane tworzenie nowych obiektów węzłów.

```
#include <NodeFactory.h>
```

Static Public Member Functions

- static Node * createNode (int val)

  Statyczna metoda tworząca nowy węzeł na stercie.

3.3.1 Detailed Description

Klasa odpowiedzialna za kontrolowane tworzenie nowych obiektów węzłów.

3.3.2 Member Function Documentation

3.3.2.1 createNode()

```
Node * NodeFactory::createNode (
            int val)  [static]
```

Statyczna metoda tworząca nowy węzeł na stercie.

Parameters

val	Wartość dla nowego węzła.

Returns

Wskaźnik do utworzonego obiektu Node.

The documentation for this class was generated from the following files:

- NodeFactory.h
- NodeFactory.cpp

Chapter 4

File Documentation

4.1 DoublyLinkedList.cpp File Reference

Implementacja pełnych operacji dla klasy DoublyLinkedList.

```
#include "DoublyLinkedList.h"
#include "NodeFactory.h"
#include <iostream>
```

4.1.1 Detailed Description

Implementacja pełnych operacji dla klasy DoublyLinkedList.

4.2 DoublyLinkedList.h File Reference

Definicja klasy zarządzającej listą dwukierunkową wraz ze wszystkimi operacjami.

```
#include "Node.h"
```

Classes

- class DoublyLinkedList

  Klasa obsługująca pełny zestaw operacji na liście dwukierunkowej.

4.2.1 Detailed Description

Definicja klasy zarządzającej listą dwukierunkową wraz ze wszystkimi operacjami.

4.3 DoublyLinkedList.h

Go to the documentation of this file.

```
00001 #ifndef DOUBLY_LINKED_LIST_H
00002 #define DOUBLY_LINKED_LIST_H
00003
00004 #include "Node.h"
00005
00010
00015 class DoublyLinkedList {
00016 private:
00017     Node* head;
00018     Node* tail;
00019
00020 public:
00021     DoublyLinkedList();
00022     ~DoublyLinkedList();
00023
00024     void prepend(int val);                          // Dodaj na początek[cite: 4]
00025     void append(int val);                           // Dodaj na koniec[cite: 4]
00026     void insertAtIndex(int index, int val);         // Dodaj pod indeks[cite: 4]
00027     void removeHead();                              // Usuń z początku[cite: 4]
00028     void removeTail();                              // Usuń z końca[cite: 4]
00029     void removeAtIndex(int index);                  // Usuń z indeksu[cite: 4]
00030     void displayForward();                          // Wyświetl całą listę[cite: 4]
00031     void displayBackward();                         // Wyświetl w odwrotnej kolejności[cite:
      4]
00032     void displayNext(Node* node);                   // Wyświetl następny element[cite: 4]
00033     void displayPrev(Node* node);                   // Wyświetl poprzedni element[cite: 4]
00034     void clear();                                   // Czyść całą listę[cite: 4]
00035
00036     Node* getNode(int index);                       // Pomocnicza: pobierz węzeł pod indeksem
00037 };
00038
00039 #endif // DOUBLY_LINKED_LIST_H
```

4.4 main.cpp File Reference

Kompleksowe testy wszystkich wymaganych operacji na liście.

```
#include <iostream>
#include "DoublyLinkedList.h"
```

Functions

- int **main** ()

4.4.1 Detailed Description

Kompleksowe testy wszystkich wymaganych operacji na liście.

4.5 Node.cpp File Reference

Implementacja konstruktora klasy Node.

```
#include "Node.h"
```

4.5.1 Detailed Description

Implementacja konstruktora klasy Node.

4.6 Node.h File Reference

Definicja klasy Node reprezentującej pojedynczy element listy dwukierunkowej.

Classes

- class Node

  Klasa przechowująca wartość oraz wskaźniki na sąsiednie węzły.

4.6.1 Detailed Description

Definicja klasy Node reprezentującej pojedynczy element listy dwukierunkowej.

4.7 Node.h

Go to the documentation of this file.

```
00001 #ifndef NODE_H
00002 #define NODE_H
00003
00008
00013 class Node {
00014 public:
00015     int data;
00016     Node* next;
00017     Node* prev;
00018
00023     Node(int val);
00024 };
00025
00026 #endif // NODE_H
```

4.8 NodeFactory.cpp File Reference

Implementacja metod klasy NodeFactory.

```
#include "NodeFactory.h"
```

4.8.1 Detailed Description

Implementacja metod klasy NodeFactory.

4.9 NodeFactory.h File Reference

Definicja klasy NodeFactory implementującej wzorzec projektowy Factory.

```
#include "Node.h"
```

Classes

- class NodeFactory

  Klasa odpowiedzialna za kontrolowane tworzenie nowych obiektów węzłów.

4.9.1 Detailed Description

Definicja klasy NodeFactory implementującej wzorzec projektowy Factory.

4.10 NodeFactory.h

Go to the documentation of this file.

```
00001 #ifndef NODE_FACTORY_H
00002 #define NODE_FACTORY_H
00003
00004 #include "Node.h"
00005
00010
00015 class NodeFactory {
00016 public:
00022     static Node* createNode(int val);
00023 };
00024
00025 #endif // NODE_FACTORY_H
```

Index

createNode
    NodeFactory, 7

DoublyLinkedList, 5
DoublyLinkedList.cpp, 9
DoublyLinkedList.h, 9

main.cpp, 10

Node, 5
    Node, 6
Node.cpp, 10
Node.h, 11
NodeFactory, 6
    createNode, 7
NodeFactory.cpp, 11
NodeFactory.h, 12